

IKE-Flow: A Branching and Release Model for STAMP-Versioned Knowledge Management

Knowledge graphs unlock capabilities impossible with text-based version control. Their explicit structure enables semantic-aware versioning, configuration management, and collaborative workflows where changes are declared rather than inferred, conflicts are detected through automated reasoning rather than textual overlap, and consistency validation leverages description logic reasoners rather than manual review. IKE-Flow^[1] builds on a coordinated system of declarative coordinates—including STAMP (Status, Time, Author, Module, Path), Language, Logic, and Navigation—integrated through **View Coordinates** that define complete, reproducible configurations.

This document presents IKE-Flow as a discussion framework rather than a prescriptive specification. It demonstrates how the formal structure of knowledge graphs—far from being a constraint—provides the foundation for scalable knowledge curation combining human expertise, computational agents, and statistical process control. The layered **Starter Set** architecture enables composable knowledge foundations, while multiple alternatives for critical design decisions accommodate diverse organizational requirements. As AI-assisted authoring becomes prevalent, IKE-Flow’s explicit semantics become essential infrastructure for maintaining quality at scale.

Table of Contents

1. Introduction	4
1.1. The Power of Explicit Structure	4
1.2. Scalable Knowledge Curation Through Formal Structure	4
1.3. Document Scope	5
1.4. Intended Audience	5
2. The Knowledge Graph Advantage	6
2.1. From Inference to Declaration	6
2.2. Capabilities Enabled by Formal Structure	6
2.3. Comparison with Text-Based Version Control	7
3. Lessons Learned from GitFlow	8
3.1. GitFlow’s Contributions	8
3.2. GitFlow Pain Points and IKE-Flow Responses	8
4. Core Concepts	10
4.1. The Minimal Component Model	10

4.2. Coordinate Types	11
4.2.1. STAMP Coordinate	11
4.2.2. Language Coordinate	11
4.2.3. Logic Coordinate	11
4.2.4. Navigation Coordinate	12
4.3. View Coordinates	12
4.3.1. Time-on-Path Within View Coordinates	12
4.4. Reified View Coordinates	13
4.4.1. Release Lifecycle Through Semantic Attachment	14
4.4.2. Reproducibility Guarantee	15
5. Scalable Curation Architecture	16
5.1. The Multi-Source Authoring Reality	16
5.2. Quality Metrics Enabled by Explicit Structure	16
5.3. Statistical Process Control	17
5.4. Reliability Tracking	19
6. Starter Set Architecture	21
6.1. Architectural Principles	21
6.2. Primordial Starter Set	21
6.3. Foundation Model Starter Sets	22
6.3.1. Status Model Deep Dive	22
6.4. Domain Starter Sets	23
6.5. Integration Sets	23
6.6. Best Practices for Starter Set Development	23
7. Path Types and Workflow Patterns	25
7.1. Path Characteristics	25
7.2. Standard Path Patterns	25
7.3. Collaborative Workflow Patterns	26
7.3.1. Feature Development Pattern	26
7.3.2. Release Pattern	26
7.3.3. Release Lifecycle Through Reified View Coordinates	27
7.3.4. Hotfix Pattern	27
8. Content Withdrawal	28
8.1. The Withdrawal Problem	28
8.2. Withdrawal Requirements	28
8.3. Withdrawal Approach Alternatives	29
8.3.1. Option A: Status-Based Withdrawal	29
8.3.2. Option B: Module-Based Withdrawal	30
8.3.3. Option C: Path-Based Withdrawal	31
8.3.4. Option D: Semantic-Based Withdrawal	31
8.4. Withdrawal Decision Framework	32
8.5. Withdrawal Cascade Considerations	32

9. IKE-Flow Conceptual Model	34
9.1. Operations Summary	34
10. Decision Points	36
10.1. Decision: Status Value Extension	36
10.2. Decision: Merge Strategy Default	36
10.3. Decision: Future Coordinate Types	37
10.4. Decision: AI-Assisted Authoring Review Policy	37
11. Example Use Cases	38
11.1. Use Case 1: National Extension Development	38
11.2. Use Case 2: Knowledge Integration	38
11.3. Use Case 3: Release Progression Through Stages	39
11.4. Use Case 4: Withdrawal of Erroneous Content	40
11.5. Use Case 5: LLM-Assisted Authoring with Quality Control	40
12. Implementation Considerations	42
12.1. Architectural Layers	42
12.2. Implementation Priorities	44
13. Open Questions	45
14. Glossary	46
15. References	48
Bibliography	48
16. Document History	49

1. Introduction

This document proposes IKE-Flow, a workflow model for managing versioned knowledge graphs. Where traditional source code version control must infer changes from textual differences, knowledge graphs declare changes explicitly through their formal structure. This fundamental difference transforms versioning and configuration management from a problem of text management into an opportunity for semantic automation and computer-supported collaborative work.

IKE-Flow draws inspiration from GitFlow^[2]'s structured branching model while recognizing that knowledge graphs offer capabilities that text-based systems cannot match. The explicit component identity, formal semantic relationships, and amenability to automated reasoning that characterize knowledge graphs are not constraints to accommodate—they are foundations to build upon.

The model introduces **View Coordinates** as complete configuration specifications integrating multiple coordinate dimensions, **Reified View Coordinates** as persistent, identity-bearing configurations that can progress through release lifecycles, and addresses the complex problem of content withdrawal through multiple alternative approaches.

1.1. The Power of Explicit Structure

Traditional version control systems face an inherent limitation: they must infer semantic meaning from textual changes. When two developers modify the same file, the system can detect overlapping lines but cannot determine whether the changes are logically compatible. Merge tools rely on heuristics and human judgment to resolve conflicts that may or may not represent actual semantic disagreements.

Knowledge graphs eliminate this inferential gap. The underlying data model^[3] ensures that every component—concept, relationship, axiom, description—has explicit identity. Changes are not diffs to be interpreted; they are assertions with precise semantics. This explicitness enables:

- **Automated consistency validation:** Description logic reasoners determine logical compatibility without human intervention
- **Precise conflict detection:** Conflicts are semantic contradictions, not textual overlaps
- **Intrinsic provenance:** Author, time, and module are part of every assertion, not metadata added after the fact
- **Machine-readable metrics:** Statistical analysis of contribution patterns becomes possible because contributions are structured data

1.2. Scalable Knowledge Curation Through Formal Structure

Modern knowledge management faces an unprecedented challenge: curating structured knowledge at scale in ways that combine human experts, computational agents, statistical process control, machine learning, and other advanced technologies. The increasing capabilities of large language models (LLMs) and AI-assisted authoring tools create both opportunities and demands that text-

based systems cannot meet.

The formal structure of knowledge graphs provides the foundation for scalable curation:

- **Inter-rater reliability assessment:** Because contributions are structured, agreement between different authors (human or computational) can be measured precisely
- **Intra-rater reliability tracking:** Consistency of individual contributors over time becomes quantifiable
- **Statistical process control:** Quality metrics can be computed, tracked, and used to calibrate review intensity
- **Source-aware processing:** Contributions from different sources can be evaluated with appropriate scrutiny based on measured reliability

Brute-force manual review cannot scale to meet modern demands. The explicit structure of knowledge graphs enables statistical processes to help scale safely while maintaining governance rigor.

1.3. Document Scope

This document addresses the following concerns:

1. **Versioning and Configuration Management:** How configurations are identified, managed, and referenced over time
2. **Starter Set Architecture:** How foundational building blocks compose into domain-specific knowledge bases
3. **Collaborative Workflow Patterns:** How content moves through development, review, and release stages with computer-supported collaboration
4. **Withdrawal Handling:** How erroneous content is managed after publication
5. **Release Management:** How stable configurations become publishable artifacts
6. **Quality at Scale:** How statistical methods and automation support scalable curation

1.4. Intended Audience

This document targets:

- Knowledge engineers designing knowledge management systems
- Software architects implementing STAMP-versioned knowledge graph capabilities
- Standards bodies considering versioning and configuration management policies
- Development teams building and extending STAMP-versioned knowledge graph capabilities
- Quality assurance specialists developing curation workflows
- Organizations integrating AI-assisted authoring into knowledge development

2. The Knowledge Graph Advantage

This section articulates why knowledge graphs provide a superior substrate for versioned knowledge management. Understanding these advantages is essential to appreciating how IKE-Flow differs from—and improves upon—text-based version control.

2.1. From Inference to Declaration

The fundamental advantage of knowledge graphs over text files is the shift from inferred to declared changes. [Table 1](#) contrasts these approaches.

Table 1. Inferred vs. Declared Change Detection

Aspect	Text-Based (Inferred)	Knowledge Graph (Declared)
Change identification	Diff algorithms compare character sequences	Components have explicit identity; changes are assertions
Conflict detection	Overlapping line ranges trigger conflicts	Logical inconsistency detected by reasoner
Semantic understanding	Tools cannot distinguish meaningful from cosmetic changes	All changes have explicit semantics
Merge resolution	Heuristics and human judgment	Automated reasoning with human review for genuine conflicts
Provenance	Commit metadata attached externally	Author, time, module intrinsic to every assertion
Quality metrics	Limited to commit counts, line changes	Structured data enables sophisticated statistical analysis

This shift has profound implications. When changes are declared rather than inferred, automated systems can reason about them. Consistency validation becomes possible. Quality metrics become meaningful. Scalable curation becomes achievable.

2.2. Capabilities Enabled by Formal Structure

The formal structure of knowledge graphs enables capabilities that text-based systems cannot provide. [Table 2](#) enumerates these capabilities.

Table 2. Capabilities Enabled by Knowledge Graph Structure

Capability	Explanation
Consistency validation	Description logic reasoners validate logical consistency before merge; inconsistent changes are identified automatically
Semantic conflict detection	Conflicts are logical contradictions (unsatisfiable concepts, contradictory axioms), not textual overlaps

Capability	Explanation
Automated redundancy detection	Reasoners identify when new content is logically equivalent to existing content
Precise impact analysis	Formal relationships enable computation of what is affected by a proposed change
Queryable history	Coordinates enable queries like "state of concept X at time T on path P"
Reliability measurement	Structured contributions enable inter-rater and intra-rater reliability statistics
Statistical process control	Quality metrics can be computed, charted, and used to detect drift
AI-assisted curation	Machine-readable structure enables LLM integration with measurable quality

2.3. Comparison with Text-Based Version Control

[Table 3](#) provides a comprehensive comparison between knowledge graph versioning and configuration management versus traditional text-based version control.

Table 3. Knowledge Graph vs. Text-Based Version Control

Aspect	Text-Based (Git)	Knowledge Graph (IKE-Flow)
Atomic unit	Lines, files	Components (concepts, semantics, patterns)
Change semantics	Implicit (inferred from diff)	Explicit (declared assertions)
Merge validation	Syntactic (no overlapping edits)	Semantic (logically consistent)
Conflict definition	Textual overlap	Logical inconsistency
Validation timing	Post-merge (CI/CD)	Pre-merge (consistency validation)
Provenance model	External metadata (commit info)	Intrinsic (STAMP coordinates)
Quality measurement	Coarse (commits, lines changed)	Fine-grained (per-component, per-author)
Scalability approach	More reviewers, better tooling	Statistical process control, calibrated review

3. Lessons Learned from GitFlow

While knowledge graphs provide advantages over text-based systems, GitFlow introduced valuable workflow concepts that IKE-Flow adapts. Understanding GitFlow’s successes and pain points informs IKE-Flow’s design.

3.1. GitFlow’s Contributions

GitFlow popularized several important concepts:

1. **Structured branching:** Named branch types (feature, release, hotfix) with defined lifecycles
2. **Release stabilization:** Dedicated branches for release qualification
3. **Parallel development:** Feature isolation enabling concurrent work
4. **Hotfix workflows:** Patterns for addressing issues in released content

These concepts translate well to knowledge graph versioning and configuration management, though the implementation differs significantly.

3.2. GitFlow Pain Points and IKE-Flow Responses

Real-world GitFlow usage revealed several pain points. [Table 4](#) presents these issues and how IKE-Flow’s knowledge graph foundation addresses them.

Table 4. GitFlow Pain Points and IKE-Flow Responses

Pain Point	Description	IKE-Flow Response
Long-lived branch divergence	Feature branches existing for weeks diverge significantly, making merges painful	Consistency validation provides early warning; semantic merge preview shows conflicts before they accumulate
Release branch proliferation	Multiple active release branches create maintenance burden	Reified View Coordinates decouple release identity from branch structure; stabilization is time-bounded
Merge ordering issues	Concurrent feature completions create merge train problems	Dependency-aware merge orchestration; explicit module boundaries
Environment coupling	Branches become coupled to deployment environments	Coordinates specify views, not deployment targets; separation of configuration from deployment
Rigid versioning	SemVer requirements don’t fit all release patterns	Time-on-path as primitive; names provide semantic identity separate from ordering

Pain Point	Description	IKE-Flow Response
Hotfix complexity	Hotfixes must merge to both main and develop, risking divergence	Path-based isolation with explicit promotion policies
Implicit change inference	Tools must infer what changed from text diffs, leading to false conflicts	Explicit component identity eliminates inference; every change is precisely identified

The last point—implicit change inference—is the root cause of many GitFlow pain points. Because text-based systems cannot understand the semantics of changes, they produce false positive conflicts (textual overlap without semantic conflict) and miss true conflicts (semantic contradiction without textual overlap). Knowledge graphs eliminate this category of problems entirely.

4. Core Concepts

This section establishes the foundational concepts upon which IKE-Flow builds. These concepts form a layered architecture where each layer depends only on layers below it.

4.1. The Minimal Component Model

IKE-Flow builds on a deliberately minimal set of component types, which provides flexibility and consistency:

- **Concepts:** Identity and meaning; the "what" being represented
- **Patterns:** Structure definitions; templates for organizing information
- **Semantics:** Meaningful extensions; add context and relationships to concepts
- **Stamps:** Version coordinates; the five STAMP dimensions

This minimal model has profound implications:

1. **No need to invent new basic types:** New kinds of knowledge are represented through Semantics extending Concepts, not through schema changes
2. **Dynamic field definition:** Fields within Semantics are defined through concepts representing data type, meaning, and purpose
3. **No compile cycles:** Changes to the knowledge model don't require system rebuilds
4. **Emergent knowledge:** Semantic integration and reasoning over new content emerges naturally from the formal structure

Figure 1 illustrates the relationships between these component types.

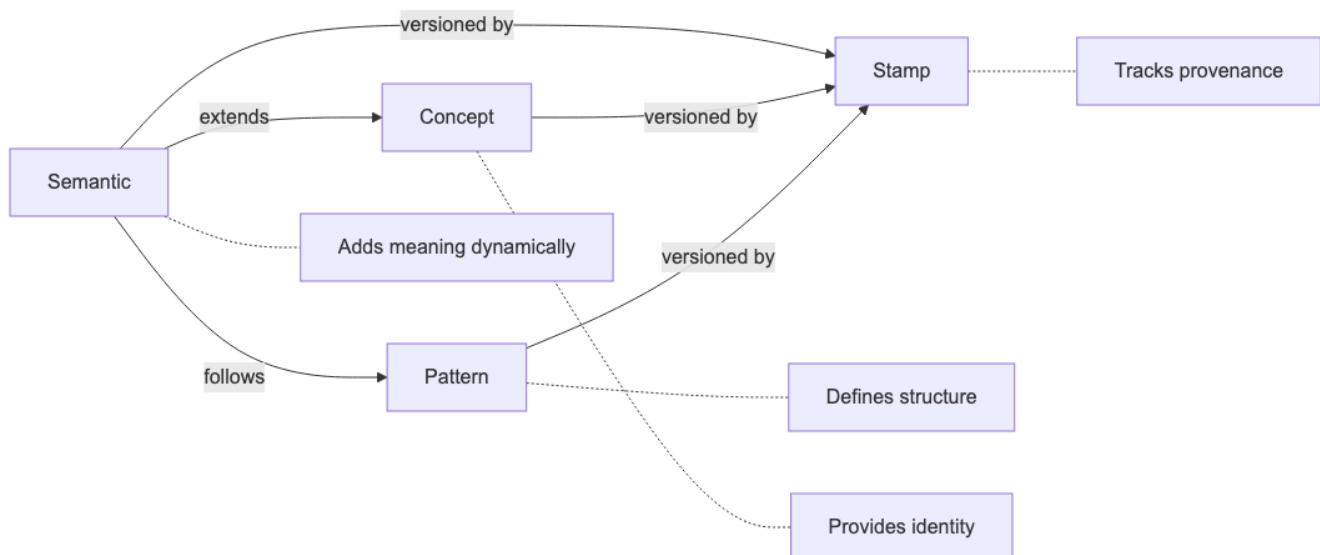


Figure 1. Minimal Component Model

4.2. Coordinate Types

IKE-Flow uses multiple coordinate types to specify different aspects of a configuration. Each coordinate type addresses a distinct concern; together they define a complete view of the knowledge graph.

4.2.1. STAMP Coordinate

The STAMP coordinate provides five dimensions for identifying any component state:

- **Status:** The lifecycle state of the component (Active, Inactive, etc.)
- **Time:** When this state was recorded (a specific instant, or LATEST for current state)
- **Author:** Who made this assertion
- **Module:** Which logical grouping contains this component
- **Path:** Which development line contains this state

Figure 2 illustrates these dimensions.

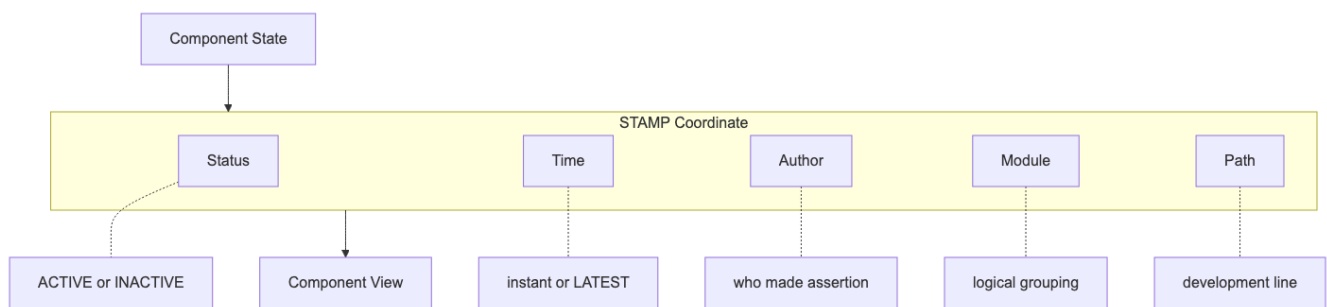


Figure 2. STAMP Coordinate Dimensions

The STAMP coordinate provides intrinsic provenance—every assertion carries its author, time, and context as part of its identity, not as external metadata.

4.2.2. Language Coordinate

The Language Coordinate specifies which natural languages are included in a view:

- Which language codes are in scope (en, es, fr, etc.)
- Dialect preferences (en-US vs. en-GB)
- Description type preferences (fully specified name, synonym, etc.)

This enables a single knowledge graph to support multilingual content while allowing views to filter to specific language needs.

4.2.3. Logic Coordinate

The Logic Coordinate specifies the description logic configuration:

- Which stated axioms are included

- Which reasoner to use for inference
- Whether to include inferred relationships
- Classification parameters

This enables different views to use different reasoning configurations—for example, a view for editing might show only stated relationships, while a view for clinical use might include inferred relationships.

4.2.4. Navigation Coordinate

The Navigation Coordinate specifies how the knowledge graph is traversed and presented:

- Which hierarchies to use for navigation
- Whether to use stated or inferred relationships for hierarchy
- Root concepts for navigation trees
- Filtering criteria for navigation

4.3. View Coordinates

A **View Coordinate** integrates all coordinate types into a complete configuration specification. [Figure 3](#) illustrates this integration.

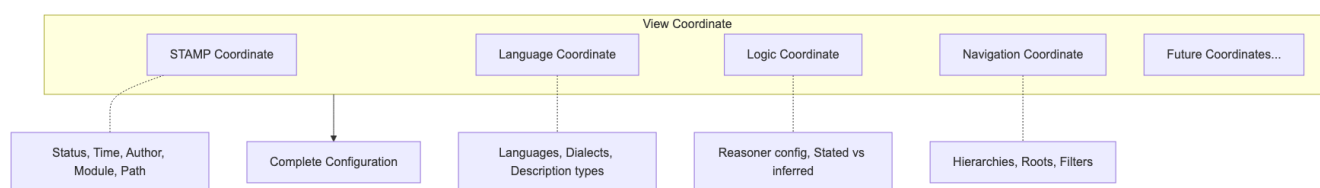


Figure 3. View Coordinate Structure

The View Coordinate provides:

- **Complete specification:** All parameters needed to define a reproducible view
- **Composability:** Each coordinate type can be varied independently
- **Extensibility:** Future coordinate types can be added without breaking existing configurations

View Coordinates can be used transiently—for a query, an interactive exploration, or a temporary analysis—without being persisted to the knowledge graph. This supports ad-hoc usage without creating unnecessary overhead.

4.3.1. Time-on-Path Within View Coordinates

Within the STAMP coordinate, **time-on-path** serves as the fundamental versioning primitive. This approach provides several benefits articulated in [Table 5](#).

Table 5. Benefits of Time-on-Path Versioning

Benefit	Explanation
Native to STAMP	No translation layer needed; the version is the coordinate
Unified snapshot/release semantics	time=LATEST provides snapshot behavior; fixed time provides stable configuration
Queryable history	Any historical state is retrievable by specifying the appropriate time
Multi-artifact coherence	Multiple artifacts can share the same time coordinate, ensuring consistency
No "breaking change" debates	The system doesn't need to determine major vs. minor—time simply advances
Quality trend analysis	Time-based versioning enables statistical analysis of quality metrics over time

This model mirrors familiar concepts from Maven^[4]'s versioning:

- A coordinate with **time=LATEST** behaves like a Maven SNAPSHOT—it represents the current head of development and may change
- A coordinate with a fixed time behaves like a Maven release—it represents an immutable, reproducible state

4.4. Reified View Coordinates

When a View Coordinate needs permanence, identity, and the ability to participate fully in the knowledge graph, it must be **reified**. Reification—from the Latin **res** (thing) and **facere** (to make)—means making an abstraction concrete.

A **Reified View Coordinate** is a View Coordinate specification that has been anchored to a Concept within the knowledge graph. [Figure 4](#) illustrates this structure.

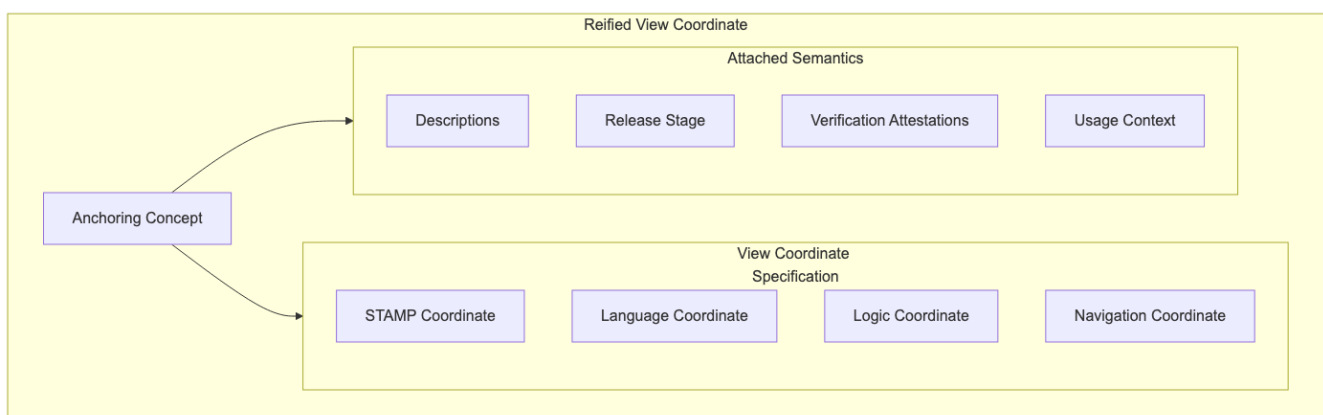


Figure 4. Reified View Coordinate Structure

Reification provides:

- **Permanent identity:** A universally unique identifier via the anchoring Concept
- **Descriptions:** Human-readable names and documentation attached as Semantics

- **Semantic attachment:** Release lifecycle stages, verification attestations, usage contexts
- **Queryability:** Discoverable through semantic criteria
- **Referenceability:** Other components can reference this configuration

4.4.1. Release Lifecycle Through Semantic Attachment

Traditional release promotion typically requires multiple repositories:

```
repo-dev → repo-staging → repo-production
```

With Reified View Coordinates, promotion becomes **adding semantics**. The View Coordinate specification remains immutable; only the attached semantics change:

```
Reified View Coordinate: "snomed-us-2025Q1"
|
|—— Pattern: View Coordinate specification (fixed)
|
|—— Semantic: ReleaseStage = "candidate" (added 2025-03-01)
|
|—— Semantic: VerificationAttestation (added 2025-03-10)
|      verifiedBy: QA-team
|      criteria: regression-suite-passed
|
|—— Semantic: ReleaseStage = "staging" (added 2025-03-12)
|
|—— Semantic: VerificationAttestation (added 2025-03-15)
|      verifiedBy: clinical-review-board
|      criteria: clinical-accuracy-review
|
|—— Semantic: ReleaseStage = "production" (added 2025-03-20)
```

This approach enables:

- **Single repository model:** No need for multiple repos for promotion stages
- **Queryable release lifecycle:** Find releases by semantic criteria, not repo location
- **Built-in audit trail:** All attestations are semantics with their own STAMP (who, when)
- **Flexible verification workflows:** Organizations define their own verification semantics
- **Append-only integrity:** Promotion adds semantics; nothing is modified or moved

[Table 6](#) presents examples of Reified View Coordinates.

Table 6. Reified View Coordinate Examples

Name	Time (STAMP)	Release Stage	Purpose
snomed-us-2025Q1-SNAPSHOT	LATEST	development	Active development snapshot
snomed-us-2025Q1-RC1	2025-03-01T00:00:00Z	candidate	Release candidate for testing
snomed-us-2025Q1	2025-03-15T00:00:00Z	production	Published quarterly release
diabetes-feature-preview	LATEST	development	Feature branch preview

4.4.2. Reproducibility Guarantee

The append-only nature of the underlying representation provides a crucial guarantee: **when a View Coordinate has no LATEST components, it defines a complete and reproducible configuration.**

This means:

- A Reified View Coordinate with all fixed values will always resolve to the same content
- Releases are inherently reproducible without special archiving mechanisms
- Historical states can be reconstructed by specifying the appropriate time coordinates

5. Scalable Curation Architecture

The explicit structure of knowledge graphs enables curation at scales impossible with text-based systems. This section presents IKE-Flow’s approach to scalable curation, leveraging the formal substrate to support quality assurance through statistical methods and computer-supported collaborative work.

5.1. The Multi-Source Authoring Reality

Modern knowledge development increasingly involves multiple types of contributors. [Table 7](#) characterizes these contributor types and their quality considerations.

Table 7. Contributor Types in Modern Knowledge Authoring

Contributor Type	Characteristics	Quality Considerations
Human domain experts	Deep knowledge, slow throughput, expensive, variable consistency	Gold standard for complex decisions; subject to fatigue, bias
Human knowledge engineers	Process expertise, moderate throughput, trained in standards	High reliability for routine work; may lack domain depth
LLM-assisted authoring	High throughput, broad knowledge, requires validation	Useful for drafts and suggestions; hallucination risk, requires review
Rule-based automation	Deterministic, high throughput, limited scope	Highly reliable within scope; brittle at boundaries
ML-based systems	Pattern recognition, probabilistic output, trainable	Good for triage and flagging; confidence calibration critical

The formal structure of knowledge graphs enables workflows where content from all these sources flows through appropriate validation, where quality metrics track reliability by source, and where statistical methods identify issues before they propagate.

5.2. Quality Metrics Enabled by Explicit Structure

Because knowledge graph contributions are structured data rather than text changes, sophisticated quality metrics become computable. [Table 8](#) presents key metrics that IKE-Flow supports.

Table 8. Quality Metrics for Knowledge Curation

Metric	Definition	Application
Inter-rater reliability	Agreement between different authors on the same content (Cohen’s kappa ^[5] , Fleiss’ kappa)	Identifying concepts needing expert adjudication; training needs assessment
Intra-rater reliability	Consistency of individual contributors over time	Detecting fatigue, drift, or changing understanding

Metric	Definition	Application
Source-specific error rate	Proportion of contributions from a source requiring correction	Calibrating review intensity by source; identifying problematic pipelines
Reasoning delta	Changes to inferred hierarchy resulting from new content	Detecting unexpected semantic impacts; regression testing
Consistency violation rate	Frequency of reasoning failures by source	Source reliability assessment; process improvement targeting
Review cycle time	Time from submission to acceptance/rejection	Process efficiency; bottleneck identification

These metrics are possible because contributions have explicit structure. Text-based systems can count commits and lines changed; knowledge graph systems can measure semantic agreement and logical consistency.

5.3. Statistical Process Control

Statistical process control (SPC)^[6] applies statistical methods to monitor and control processes. In knowledge curation, SPC enables detection of quality drift and calibration of review intensity based on measured reliability rather than assumptions.

Figure 5 illustrates how statistical process control integrates with the IKE-Flow workflow.

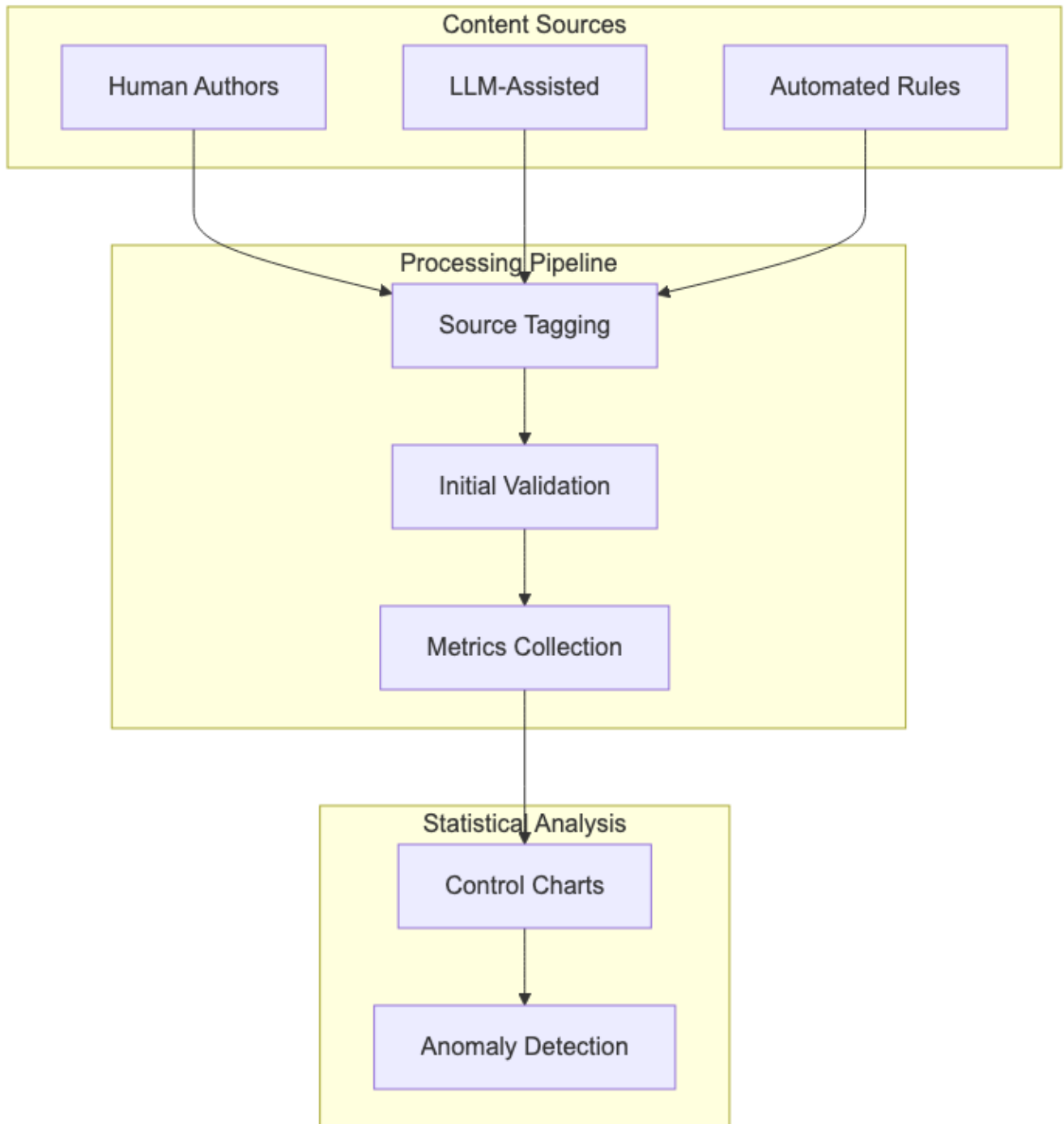


Figure 5. Statistical Process Control in IKE-Flow

Figure 6 shows how statistical analysis informs review routing decisions.

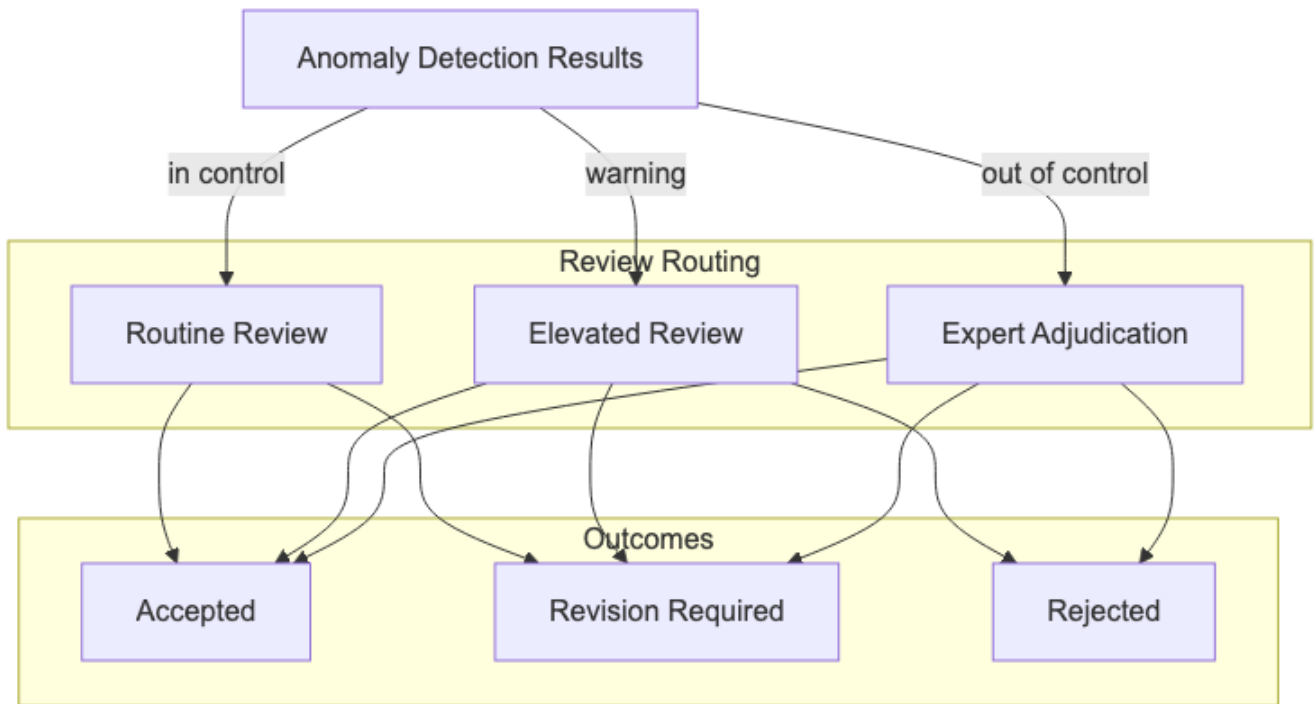


Figure 6. Review Routing Based on Statistical Analysis

The key insight is that review intensity should be calibrated by source reliability and content risk, not applied uniformly. A contribution from a source with established high reliability on routine content needs less scrutiny than novel content from a new LLM pipeline. The explicit structure of knowledge graphs makes this calibration measurable and actionable.

5.4. Reliability Tracking

IKE-Flow supports reliability tracking through patterns enabled by the formal knowledge graph structure:

1. **Source attribution:** Every component carries author information in its STAMP; this extends to computational agents, which are modeled as authors
2. **Batch identification:** Contributions can be grouped into batches for aggregate quality analysis
3. **Gold standard comparison:** Designated "gold standard" content enables calibration of new sources
4. **Temporal windowing:** Reliability metrics are computed over rolling time windows to detect drift

Figure 7 shows the data flow for reliability tracking.

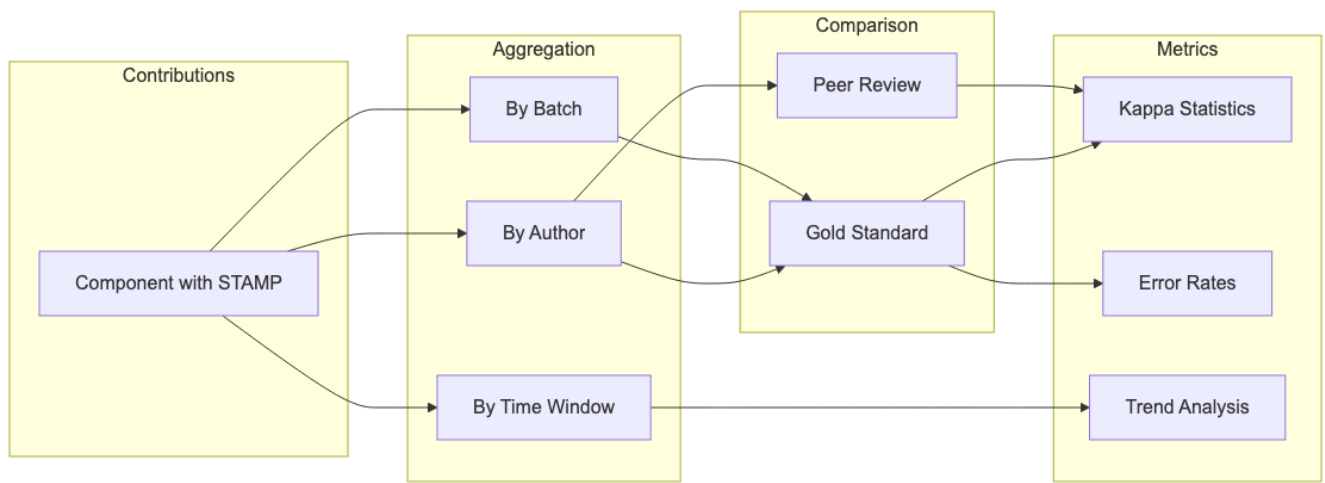


Figure 7. Reliability Tracking Data Flow

6. Starter Set Architecture

IKE-Flow introduces the concept of **Starter Sets**—foundational building blocks that provide the concepts, patterns, and infrastructure needed to build domain-specific knowledge bases. Starter Sets compose hierarchically, with each set building on the capabilities of sets below it.

6.1. Architectural Principles

The Starter Set architecture follows several guiding principles:

1. **Layered composition:** Higher-level sets depend on lower-level sets, never the reverse
2. **Separation of concerns:** Each set addresses a specific domain or capability
3. **Extension without modification:** New capabilities extend existing sets rather than modifying them
4. **Explicit dependencies:** Every set declares its dependencies on other sets

Figure 8 illustrates the layered architecture of Starter Sets.

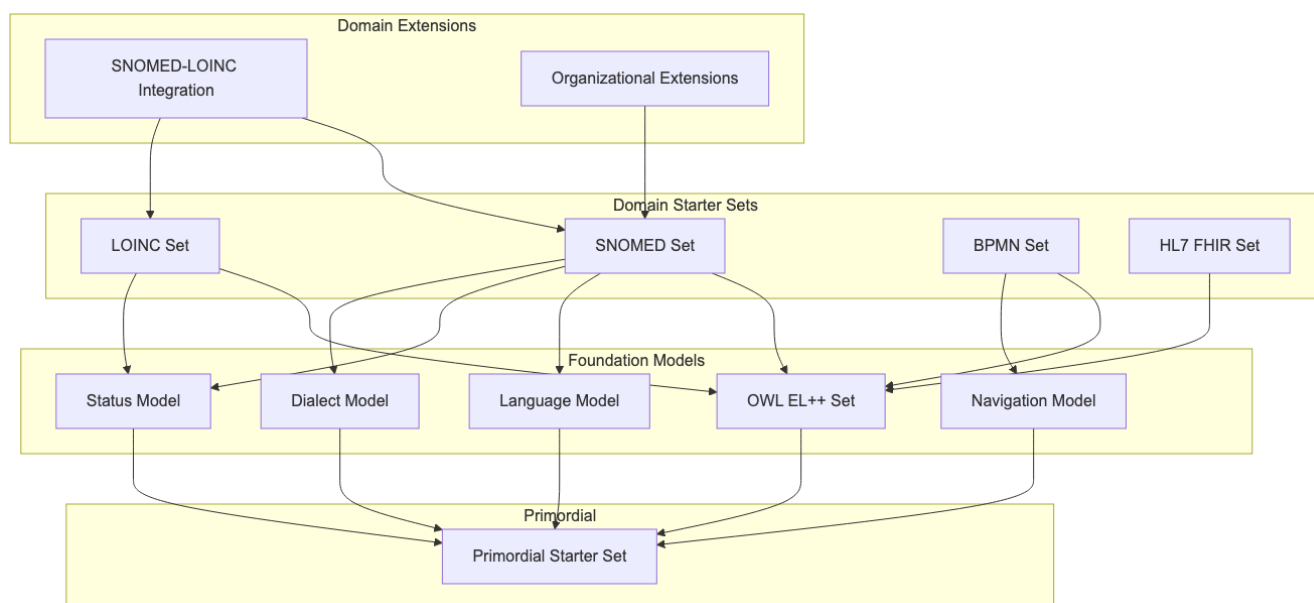


Figure 8. Starter Set Layered Architecture

6.2. Primordial Starter Set

The Primordial Starter Set provides the absolute foundation—the concepts and patterns without which no other set can function. The term "primordial" indicates these elements exist prior to any domain-specific content; they are the substrate upon which all knowledge is built.

Table 9 enumerates the categories of content in the Primordial Starter Set.

Table 9. Primordial Starter Set Contents

Category	Contents	Purpose
Identity Patterns	Concept pattern, Semantic pattern, Component identifiers	Fundamental building blocks for all knowledge representation
Version Infrastructure	Stamp Coordinate pattern, Path concepts, Module concepts	Enable STAMP-based versioning and configuration management
Coordinate Infrastructure	View Coordinate patterns, Language/Logic/Navigation coordinate patterns	Enable complete configuration specification
Metadata Patterns	Description patterns, Provenance patterns	Attach human-readable and audit information
Core Relationships	Is-a, Part-of, References	Minimal relationship vocabulary
Primitive Types	String, Integer, Instant, Boolean, UUID	Data type foundations
Quality Infrastructure	Author concepts, Source type concepts, Batch identifier patterns	Enable reliability tracking and quality metrics

6.3. Foundation Model Starter Sets

Building on the Primordial Set, Foundation Model Starter Sets provide reusable capabilities that multiple domain sets require. [Table 10](#) describes each foundation set and its role.

Table 10. Foundation Model Starter Sets

Starter Set	Description	Key Concepts
OWL EL++ Set ^[7]	Concepts representing OWL EL++ language tokens and constructs; enables description logic semantics	ObjectSomeValuesFrom, ObjectIntersectionOf, EquivalentClasses, SubClassOf, Datatype restrictions
Language Model	Concepts for natural language representation; language codes, character sets	Language concepts (en, es, fr, etc.), Script concepts, Directionality
Status Model	Lifecycle status values and their semantics; see Section 6.3.1 for detailed discussion	Active, Inactive, and potential extensions
Dialect Model	Regional language variations and acceptability; follows the pattern where dialect acceptability is a semantic referencing a description	Dialect concepts (US English, GB English), Acceptability values (Preferred, Acceptable)
Navigation Model	Concepts supporting user interface navigation and concept organization; may depend on defining vs. non-defining semantics	Navigation concepts, Hierarchy roots, View specifications

6.3.1. Status Model Deep Dive

The Status Model warrants particular attention because it intersects with the withdrawal problem discussed in [\[withdrawal-section\]](#). Currently, two status values are written to the datastore:

- **ACTIVE:** The component is current and valid for use
- **INACTIVE:** The component was valid but is now retired or superseded

The question of whether additional status values belong in the Primordial/Foundation layer or should be handled through extension mechanisms is a key design decision. [\[decision-status-extension\]](#) presents the alternatives.

6.4. Domain Starter Sets

Domain Starter Sets provide the vocabulary and semantics for specific knowledge domains. Each domain set depends on appropriate foundation sets. [Table 11](#) provides examples of domain sets and their dependencies.

Table 11. Example Domain Starter Sets

Domain Set	Dependencies	Contents
SNOMED Starter Set ^[8]	OWL EL++, Language, Status, Dialect	SNOMED CT concept model, relationship types, description types, module structure
LOINC Starter Set ^[9]	OWL EL++, Language, Status	LOINC axes, component types, property concepts, timing concepts
BPMN Starter Set ^[10]	OWL EL++, Navigation	Process, Task, Gateway, Event, Sequence Flow, Message Flow

6.5. Integration Sets

Integration Sets build on multiple domain sets to provide cross-domain capabilities. The SNOMED/LOINC Integration Set, for example, depends on both the SNOMED and LOINC Starter Sets and provides mappings, coordinated expressions, and unified views.

6.6. Best Practices for Starter Set Development

When developing a new Starter Set, teams should follow these practices:

1. **Declare dependencies explicitly:** List all Starter Sets required as dependencies; never assume implicit availability
2. **Minimize dependencies:** Depend only on what you need; avoid pulling in entire domain sets for a few concepts
3. **Version your Starter Set:** Use Reified View Coordinates to identify Starter Set releases
4. **Document extension points:** Indicate where and how the set can be extended
5. **Provide examples:** Include example usage patterns for common scenarios

[Figure 9](#) illustrates the process of developing a new Starter Set.

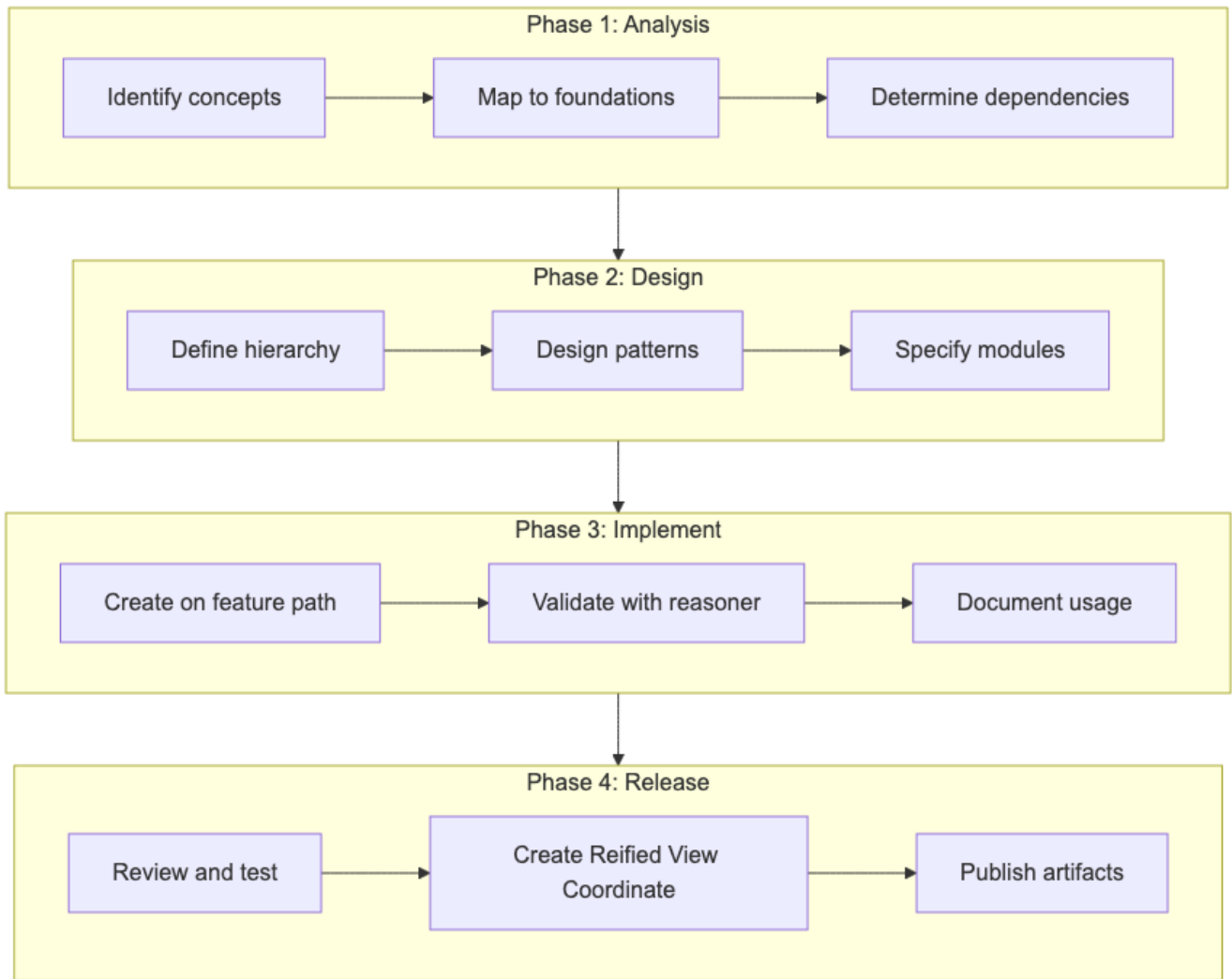


Figure 9. Starter Set Development Process

7. Path Types and Workflow Patterns

IKE-Flow defines path types not as a rigid enumeration but as a set of characteristics that can be composed. This section presents the conceptual model for paths and the collaborative workflow patterns they enable.

7.1. Path Characteristics

Rather than defining path types through a single enumeration, IKE-Flow decomposes path identity into orthogonal characteristics. This approach provides flexibility while maintaining clarity. [Table 12](#) defines these characteristics.

Table 12. Path Characteristic Dimensions

Dimension	Values	Description
Lifecycle Stage	Inception, Development, Stabilization, Released	Where in the release lifecycle this path sits
Mutability	Mutable, Append-only, Immutable	What changes are permitted on this path
Origin	External-import, Internal-authoring, Fork	How content arrives on this path
Governance	Open, Review-required, Locked	What approval process applies to changes

[Figure 10](#) visualizes how these characteristics combine to describe any path.

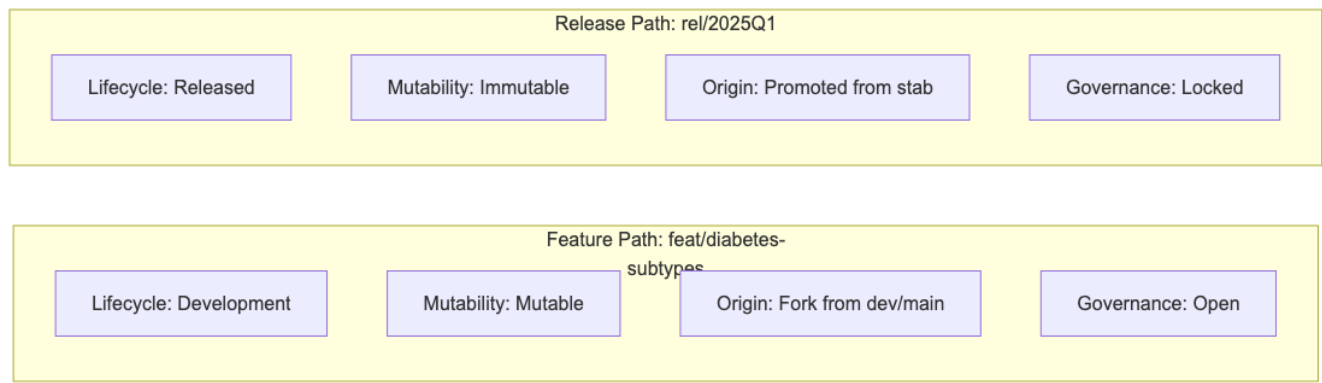


Figure 10. Path Characteristic Composition

7.2. Standard Path Patterns

While paths are flexible, certain patterns emerge as best practices. [Table 13](#) describes standard path patterns and their typical characteristics.

Table 13. Standard Path Patterns

Pattern	Lifecycle	Mutability	Governance	Purpose
primordial/*	Released	Immutable	Locked	External imports (SNOMED International, LOINC)
dev/main	Development	Mutable	Review-required	Main development line; integration target
feat/*	Inception/Development	Mutable	Open	Feature development; isolated work
stab/*	Stabilization	Append-only (fixes)	Review-required	Release candidates; bug fixes only
rel/*	Released	Immutable	Locked	Published releases; never modified

7.3. Collaborative Workflow Patterns

IKE-Flow supports several workflow patterns that leverage the knowledge graph’s ability to validate changes semantically rather than textually. These patterns emphasize computer-supported collaborative work rather than imposed constraints.

7.3.1. Feature Development Pattern

The feature development pattern isolates work on specific capabilities, validates changes through reasoning, and integrates back to the main development line.

Figure 11 shows the first phase of the feature workflow: creation and authoring.



Figure 11. Feature Workflow: Creation and Authoring

Figure 12 shows the second phase: validation and merge.



Figure 12. Feature Workflow: Validation and Merge

The feature pattern includes consistency validation at two points: during development (incremental) and before merge (full). This early validation—possible only because of the formal knowledge graph structure—prevents the accumulation of issues that plagued long-lived GitFlow feature branches.

7.3.2. Release Pattern

The release pattern creates a stabilization period during which only fixes are permitted, validates the release through reasoning, and produces immutable release artifacts.

Figure 13 shows the stabilization phase.



Figure 13. Release Workflow: Stabilization

Figure 14 shows the promotion and export phase.

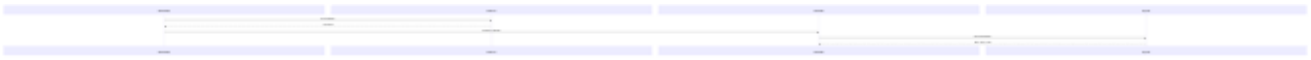


Figure 14. Release Workflow: Promotion and Export

7.3.3. Release Lifecycle Through Reified View Coordinates

Once content is on a release path, the Reified View Coordinate mechanism enables progression through verification and deployment stages without moving content between repositories.

Figure 15 illustrates this progression.

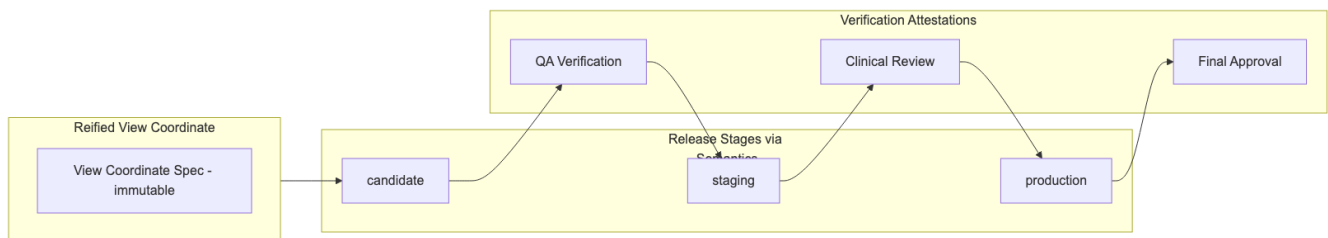


Figure 15. Release Lifecycle Through Semantic Attachment

7.3.4. Hotfix Pattern

The hotfix pattern addresses critical issues discovered after release.

Figure 16 illustrates this workflow.



Figure 16. Hotfix Workflow

8. Content Withdrawal

Content withdrawal addresses a fundamental challenge: what happens when published content should never have existed? This differs from normal retirement (where valid content becomes outdated) because withdrawn content was erroneous from the start.

8.1. The Withdrawal Problem

Withdrawal scenarios arise from several causes:

- **Intellectual property violation:** Content included copyrighted material
- **Safety error:** Content represented incorrect clinical information
- **Duplicate creation:** Content duplicated existing concepts, causing mapping errors
- **Test data exposure:** Development or test content was accidentally released
- **Governance violation:** Content bypassed required review processes

In each case, the content cannot simply be marked "inactive" because that status implies the content was once valid. Withdrawn content requires different semantics: it should be treated as if it never existed, while still maintaining audit trails for accountability.

8.2. Withdrawal Requirements

[Table 14](#) enumerates the requirements that any withdrawal approach must satisfy.

Table 14. Withdrawal Mechanism Requirements

Requirement	Description	Priority
R1: Clear semantics	The withdrawn state must be semantically distinct from inactive	Must have
R2: Query exclusion	Default queries should not return withdrawn content	Must have
R3: Audit preservation	The fact that content existed and was withdrawn must be recoverable	Must have
R4: Reason capture	The reason for withdrawal must be recorded	Must have
R5: Cascade handling	Dependent content must be addressed (see [withdrawal-cascade])	Must have
R6: Replacement linkage	If replacement content exists, the link should be capturable	Should have
R7: Temporal queries	Historical queries should be able to include or exclude withdrawn content based on policy	Should have
R8: Minimal overhead	The mechanism should not impose excessive storage or query overhead	Should have

8.3. Withdrawal Approach Alternatives

This section presents four alternative approaches to implementing withdrawal, each with distinct trade-offs. [Table 15](#) provides a summary comparison; subsequent sections elaborate on each approach.

Table 15. Withdrawal Approach Comparison Summary

Approach	Semantic Clarity	Query Impact	Audit Trail	Complexity	Industry Precedent
Status-based	High	Moderate	Preserved	Low	HL7 FHIR ^[11] , HL7 v3
Module-based	Moderate	Low	Preserved	Moderate	Limited
Path-based	Moderate	Low	Preserved	Moderate	Git (deleted branches)
Semantic-based	High	High	Rich	High	RDF deprecation patterns

8.3.1. Option A: Status-Based Withdrawal

This approach introduces **WITHDRAWN** as a new status value alongside **ACTIVE** and **INACTIVE**.

Mechanism: A component's status is changed to **WITHDRAWN**. Queries filter on status as they do today; default queries exclude **WITHDRAWN** status.

Example state transition:

```
Component: "Diabetes Type 2 (incorrect)"
Status: ACTIVE -> WITHDRAWN
Time: 2025-03-15T10:30:00Z
Author: governance-board
Reason: (captured in withdrawal semantic)
```

Advantages:

1. Explicit, unambiguous semantics
2. Consistent with existing status-based filtering
3. Minimal structural change to components
4. Easy to query ("show me all withdrawn content")

Disadvantages:

1. Expands core status enumeration (design philosophy question)
2. Requires all existing queries to consider new status

3. Status values are typically small set by design

Industry precedent: HL7 FHIR uses "entered-in-error" as a distinct status for resources that should never have existed. HL7 v3 messaging includes "nullified" for similar purposes. Both recognize that erroneous content requires different semantics than retired content.

8.3.2. Option B: Module-Based Withdrawal

This approach uses module membership to segregate withdrawn content from active content.

Mechanism: Withdrawn components are moved to a designated withdrawal module (e.g., **us-extension-withdrawn**). Views include or exclude this module based on policy.

Example state transition:

```
Component: "Diabetes Type 2 (incorrect)"
Module: us-extension -> us-extension-withdrawn
Time: 2025-03-15T10:30:00Z
Author: governance-board
```

Figure 17 illustrates how module-based withdrawal enables different views.

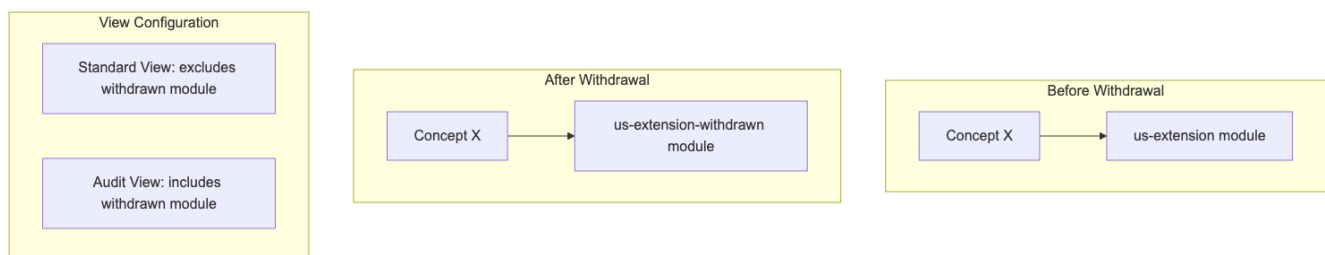


Figure 17. Module-Based Withdrawal Views

Advantages:

1. Uses existing module machinery
2. View-configurable visibility
3. Same component can be withdrawn from one module while active in another
4. No change to status model

Disadvantages:

1. Component changes module (provenance implication)
2. Module proliferation (every module needs a withdrawal counterpart)
3. "Withdrawn" is implicit in module membership rather than explicit in status

Industry precedent: Limited direct precedent, though SNOMED CT's module structure could theoretically support this pattern. More commonly seen in document management systems with "trash" or "archive" containers.

8.3.3. Option C: Path-Based Withdrawal

This approach isolates withdrawn content on dedicated paths, similar to how Git branches can be deleted (content becomes unreachable from main branches).

Mechanism: Withdrawn components are placed on a withdrawal path. Standard views do not include withdrawal paths; audit views can include them explicitly.

Example state transition:

```
Component: "Diabetes Type 2 (incorrect)"
Path: dev/us-extension -> withdrawn/us-extension-2025Q1
Time: 2025-03-15T10:30:00Z
Author: governance-board
```

Advantages:

1. Clear isolation
2. Path-based access control integrates naturally
3. Withdrawal is a "movement" operation, familiar from Git

Disadvantages:

1. May conflate path semantics (paths are development lines, not content categories)
2. Requires explicit withdrawal paths
3. Component location changes (provenance implication)

Industry precedent: Git's model where deleted branches become unreachable commits (recoverable via reflog but not visible in normal operations).

8.3.4. Option D: Semantic-Based Withdrawal

This approach attaches a withdrawal semantic to withdrawn components, following the pattern established for dialect acceptability.

Mechanism: A withdrawal semantic is created that references the withdrawn component. The semantic captures reason, date, authority, and optional replacement. Queries check for absence of withdrawal semantic (or ignore it for audit).

Example structure:

```
WithdrawalSemantic:
  references: Concept X (the withdrawn component)
  reason: IP_VIOLATION
  withdrawalDate: 2025-03-15T10:30:00Z
  authority: governance-board
  replacementComponent: (optional) Concept Y
```

Figure 18 shows the structure of semantic-based withdrawal.

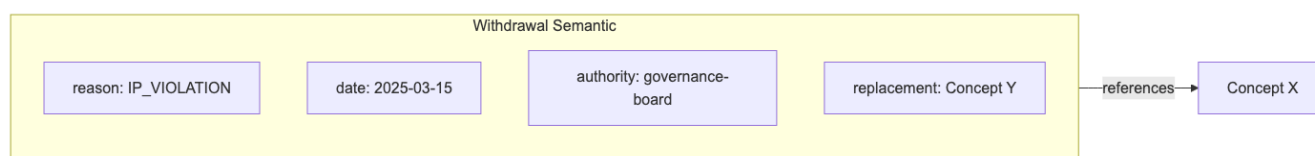


Figure 18. Semantic-Based Withdrawal Structure

Advantages:

1. Rich metadata capture
2. Follows established semantic patterns (dialect model)
3. Component itself is unchanged
4. Extensible (add fields without schema changes)

Disadvantages:

1. Query overhead (must check for semantic absence)
2. More complex to implement
3. Semantic must be checked everywhere component is used

Industry precedent: RDF/OWL^[12] deprecation often uses annotation properties. Dublin Core^[13] uses `dct:isReplacedBy`. The pattern of metadata-about-content is well established in semantic web practices.

8.4. Withdrawal Decision Framework

Table 16 presents a decision framework for selecting a withdrawal approach based on organizational priorities.

Table 16. Withdrawal Approach Decision Framework

If your priority is...	Status	Module	Path	Semantic
Semantic clarity	**	*	*	**
Minimal query changes	*	**	**	
Rich metadata				**
Alignment with FHIR	**			*
Minimal core changes		**	*	**
Simple implementation	**	*	*	

Legend: ** = strong fit, * = moderate fit, blank = weak fit

8.5. Withdrawal Cascade Considerations

When a component is withdrawn, dependent components must be addressed. Figure 19 illustrates

potential cascade scenarios.

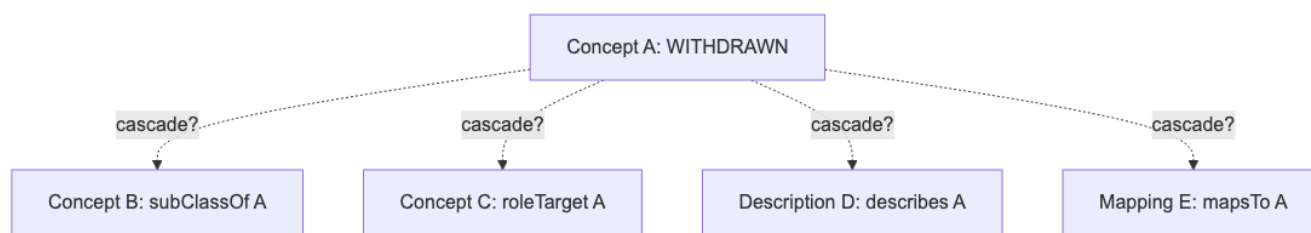


Figure 19. Withdrawal Cascade Scenarios

Table 17 presents options for handling cascade scenarios.

Table 17. Cascade Handling Options

Option	Description	Trade-off
Manual review	Each dependent is manually evaluated; no automatic cascade	Safe but labor-intensive
Relationship-type rules	Define cascade behavior by relationship type (e.g., descriptions always cascade, mappings flag for review)	Balanced; requires rule definition
Reason-based rules	Cascade behavior depends on withdrawal reason (IP issues cascade broadly; duplicates require review)	Context-sensitive; complex to implement
Full cascade	All dependents automatically withdrawn	Fast but may over-withdraw

The cascade question illustrates why the withdrawal mechanism choice matters: semantic-based withdrawal can capture cascade policy in the semantic itself; status-based withdrawal requires external policy configuration.

9. IKE-Flow Conceptual Model

This section synthesizes the concepts presented above into a complete IKE-Flow model. [Figure 20](#) presents the overall architecture.

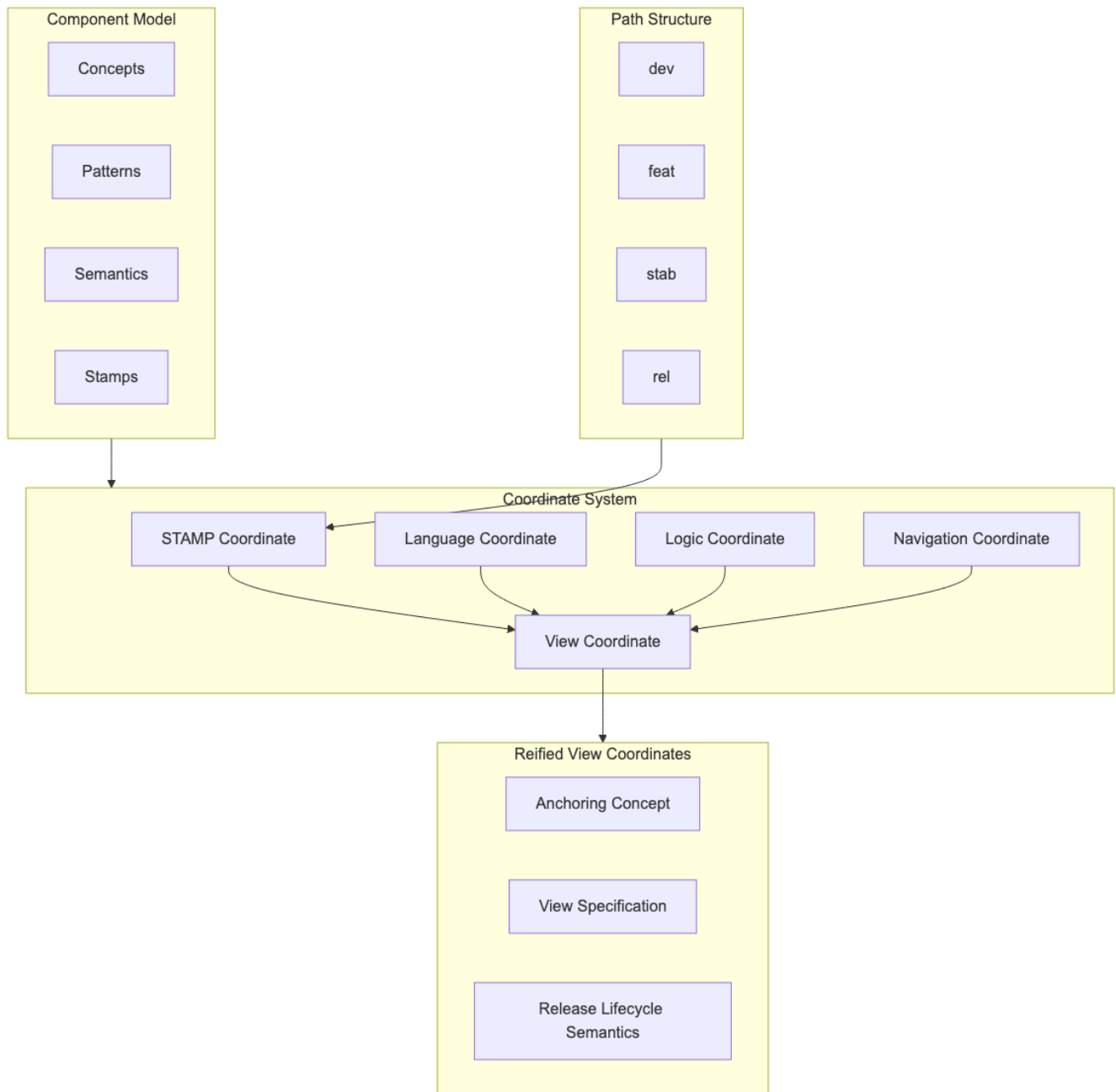


Figure 20. IKE-Flow Complete Conceptual Model

9.1. Operations Summary

[Table 18](#) summarizes the core operations in IKE-Flow.

Table 18. IKE-Flow Core Operations

Operation	Input	Output
Create Path	Source path, path name, characteristics	New path forked from source

Operation	Input	Output
Merge	Source path, target path, strategy	Merged state (if validation passes) or conflict report
Create View Coordinate	STAMP, Language, Logic, Navigation coordinates	Complete configuration specification
Reify View Coordinate	View Coordinate specification	Reified View Coordinate anchored to Concept
Attach Release Semantic	Reified View Coordinate, release stage/attestation	Updated release lifecycle
Export	Reified View Coordinate, format options	Artifacts (RF2 ^[14] , OWL, FHIR ^[15])
Withdraw	Component, reason, cascade policy	Withdrawn component (via chosen mechanism)
Compute Quality Metrics	Time window, source filter, metric type	Reliability statistics, control charts

10. Decision Points

This section consolidates key design decisions that organizations must make when adopting IKE-Flow. Each decision point presents alternatives with trade-offs.

10.1. Decision: Status Value Extension

Question: Should additional status values (such as WITHDRAWN) be part of the Primordial foundation or handled through extension mechanisms?

Option A: Primordial status values only (ACTIVE, INACTIVE)

- *Rationale:* Minimal core; status extension is rare and organization-specific
- *Implication:* Withdrawal must use module, path, or semantic approach

Option B: Extended primordial set (ACTIVE, INACTIVE, WITHDRAWN)

- *Rationale:* Withdrawal is universal need; belongs in foundation
- *Implication:* All systems must handle three-value status

Option C: Extensible status model

- *Rationale:* Organizations define their own status values; primordial provides mechanism
- *Implication:* Interoperability challenges; more complex tooling

10.2. Decision: Merge Strategy Default

Question: What should the default behavior be when merge conflicts are detected?

Option A: Block and report

- *Rationale:* Safety first; human review for all conflicts
- *Implication:* Slower throughput; requires human availability

Option B: Auto-resolve where possible

- *Rationale:* Efficiency; only truly ambiguous conflicts need humans
- *Implication:* Risk of incorrect auto-resolution; requires well-defined rules

Option C: Configurable per path

- *Rationale:* Different paths have different needs (feature=permissive, release=strict)
- *Implication:* More configuration; potential inconsistency

10.3. Decision: Future Coordinate Types

Question: How should new coordinate types be added to View Coordinates?

Option A: Fixed set of coordinates

- *Rationale:* Simplicity; all systems know all coordinate types
- *Implication:* Adding new coordinates requires system-wide updates

Option B: Extensible coordinate model

- *Rationale:* Future-proofing; organizations can add domain-specific coordinates
- *Implication:* Complexity; coordinate compatibility questions

Option C: Core coordinates plus optional extensions

- *Rationale:* Balance of stability and extensibility
- *Implication:* Need clear rules for what's core vs. extension

10.4. Decision: AI-Assisted Authoring Review Policy

Question: How should contributions from LLM-assisted authoring be reviewed?

Option A: Same review as human contributions

- *Rationale:* Source-agnostic; avoids bias
- *Implication:* May under-review high-risk AI content

Option B: Elevated review for all AI contributions

- *Rationale:* Conservative; acknowledges AI limitations
- *Implication:* May over-review reliable AI pipelines; throughput impact

Option C: Calibrated review based on source reliability metrics

- *Rationale:* Statistical basis; adapts to actual performance
- *Implication:* Requires reliability tracking infrastructure; initial calibration period

11. Example Use Cases

This section illustrates IKE-Flow concepts through concrete examples.

11.1. Use Case 1: National Extension Development

An organization develops a national extension to SNOMED CT.

Starter Set dependencies:

- SNOMED Starter Set (provides SNOMED concept model)
- Language Model (for national language)
- Dialect Model (for national dialect)

Path structure:

primordial/snomed-international-20250131	# Import of SNOMED International
dev/us-extension	# Development of US extension
feat/us-diabetes-subtypes	# Feature: additional diabetes concepts
stab/us-extension-2025Q1	# Stabilization for Q1 release
rel/us-extension-2025Q1	# Published Q1 release

Reified View Coordinates:

- **snomed-us-2025Q1-SNAPSHOT**: STAMP.time=LATEST, path=dev/us-extension
- **snomed-us-2025Q1**: STAMP.time=2025-03-15, path=rel/us-extension-2025Q1

11.2. Use Case 2: Knowledge Integration

An organization integrates SNOMED CT and LOINC.

Starter Set dependencies:

- SNOMED Starter Set
- LOINC Starter Set
- SNOMED/LOINC Integration Set (provides mapping patterns)

Path structure:

primordial/snomed-international-20250131	# SNOMED import
primordial/loinc-2.77	# LOINC import
dev/snomed-loinc-integration	# Integration development
rel/snomed-loinc-integration-2025Q1	# Published integration

Reified View Coordinate:

- **snomed-loinc-integrated-2025Q1**: STAMP.time=2025-03-15, paths=[rel/snomed-loinc-integration, primordial/snomed-international, primordial/loinc]

11.3. Use Case 3: Release Progression Through Stages

A release progresses from candidate through production.

Scenario: The **snomed-us-2025Q1** release moves through verification stages.

Initial reification:

```
Reified View Coordinate: snomed-us-2025Q1
View Specification:
  STAMP: time=2025-03-01T00:00:00Z, path=stab/us-extension-2025Q1
  Language: en-US
  Logic: OWL-EL-stated-and-inferred
  Navigation: snomed-hierarchy
```

Progression through semantics:

```
2025-03-01: Semantic attached: ReleaseStage=candidate
2025-03-05: Semantic attached: VerificationAttestation
              verifiedBy=automated-regression-suite
              result=passed
2025-03-08: Semantic attached: VerificationAttestation
              verifiedBy=qa-team
              result=passed-with-notes
2025-03-10: Semantic attached: ReleaseStage=staging
2025-03-12: Semantic attached: VerificationAttestation
              verifiedBy=clinical-review-board
              result=approved
2025-03-15: Semantic attached: ReleaseStage=production
```

Querying for releases:

```
# Find all production releases
SELECT * WHERE ReleaseStage = "production"

# Find releases verified by clinical-review-board
SELECT * WHERE VerificationAttestation.verifiedBy = "clinical-review-board"

# Find candidate releases awaiting clinical review
SELECT * WHERE ReleaseStage = "candidate"
          AND NOT EXISTS VerificationAttestation.verifiedBy = "clinical-review-board"
```

11.4. Use Case 4: Withdrawal of Erroneous Content

An IP violation is discovered in published content.

Scenario: Concept "Drug X Interaction Panel" was released in `rel/us-extension-2025Q1` but contains copyrighted content from a third party.

Using status-based withdrawal:

```
Component: "Drug X Interaction Panel"
Previous Status: ACTIVE
New Status: WITHDRAWN
Withdrawal Reason: IP_VIOLATION
Withdrawal Date: 2025-04-10
Authority: Governance Board
```

Using module-based withdrawal:

```
Component: "Drug X Interaction Panel"
Previous Module: us-extension
New Module: us-extension-withdrawn
Withdrawal Reason: (captured in semantic)
```

Cascade handling: The concept has three descriptions and two relationships targeting it. Under manual review policy, each is flagged for governance board decision.

11.5. Use Case 5: LLM-Assisted Authoring with Quality Control

An organization uses LLM assistance to accelerate knowledge development.

Scenario: An LLM pipeline generates draft concept definitions for review.

Workflow:

1. LLM generates 100 draft diabetes subtype concepts
2. Each concept is tagged with `author=llm-pipeline-v2` and `batch=diabetes-2025-03-batch-1`
3. Initial reasoning validates logical consistency (5 concepts fail, flagged for human review)
4. Remaining 95 concepts enter review queue with elevated review flag
5. Human reviewer approves 90, requests revision on 5
6. Quality metrics computed: 90% acceptance rate, 5% consistency failure rate
7. Metrics compared to control chart; pipeline is "in control"
8. Accepted concepts merged to dev/main

Quality tracking:

Source: llm-pipeline-v2

Batch: diabetes-2025-03-batch-1

Consistency pass rate: 95%

Review acceptance rate: 94.7%

Comparison to historical: within 2 sigma (in control)

12. Implementation Considerations

12.1. Architectural Layers

[Figure 21](#) presents the recommended architectural layers for implementing IKE-Flow.

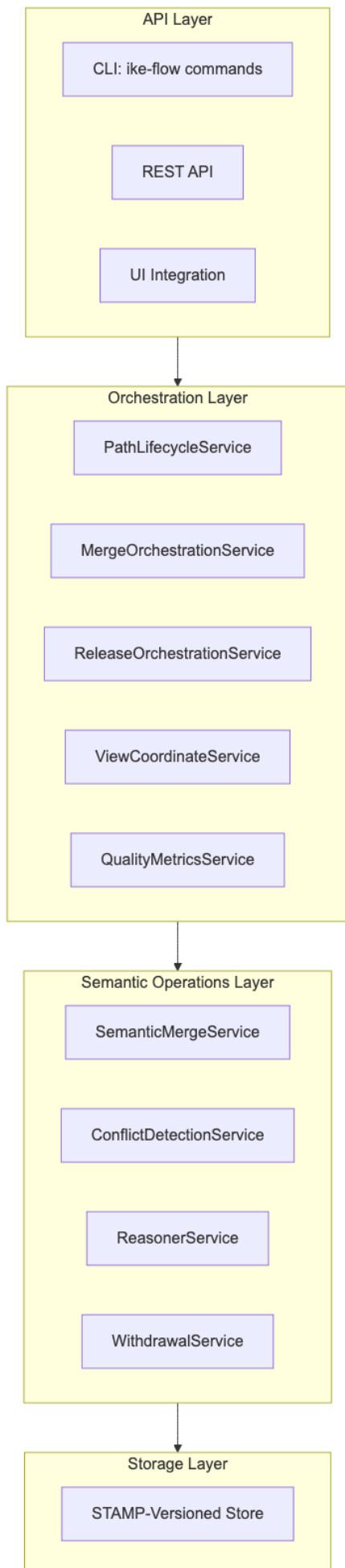


Figure 21. IKE-Flow Implementation Architecture

12.2. Implementation Priorities

Table 19 presents a recommended implementation order based on dependencies and value delivery.

Table 19. Recommended Implementation Order

Phase	Components	Rationale
1	Semantic Merge + Consistency Validation	Foundation leveraging knowledge graph's key advantage: automated semantic validation
2	Path Lifecycle Management	Basic create/fork/delete operations; enables workflow
3	View Coordinate Service	Configuration specification; enables complete views
4	Reified View Coordinate Service	Persistent configurations; enables release management
5	Conflict Detection	Semantic conflict identification; improves merge quality
6	Release Packaging (RF2, OWL)	Artifact generation; delivers value to consumers
7	Quality Metrics Service	Enables reliability tracking; supports scalable curation
8	Withdrawal Service	Addresses error correction; chosen mechanism from alternatives
9	CLI/API	Developer-facing tooling; improves usability
10	FHIR Export + CI Integration	Ecosystem integration; automation

13. Open Questions

The following questions remain open for further discussion and resolution:

1. **Module boundaries during merge:** Can a feature path span modules, or should there be one path per module? What are the implications for reasoning?
2. **Dependency ordering:** When releasing an extension that depends on core, what is the orchestration sequence? Must core release complete before extension release begins?
3. **Partial promotion:** What is the equivalent of Git cherry-pick? Can a subset of changes move between paths while maintaining provenance?
4. **Starter Set versioning:** How are Starter Sets themselves versioned? Do they follow the same Reified View Coordinate pattern?
5. **External contribution flow:** How do submissions from outside the organization enter the system? What is the path from submission to integration?
6. **Withdrawal finality:** Can a withdrawal be reversed? Under what circumstances? What does the audit trail show?
7. **Time zone handling:** STAMP uses instants, but organizations span time zones. How are release times coordinated?
8. **AI author identity:** How are different LLM versions/configurations represented as authors? What granularity is appropriate?
9. **Reliability calibration period:** How much data is needed before a new source's reliability metrics are considered stable?
10. **Cross-organization reliability:** Can reliability metrics be shared across organizations using the same Starter Sets?
11. **Coordinate compatibility:** When new coordinate types are added, how are existing Reified View Coordinates migrated or interpreted?

14. Glossary

Concepts

The foundational component type providing identity and meaning in the minimal component model.

Consistency Validation

The process of using description logic reasoners to verify that knowledge graph content is logically consistent before operations such as merge proceed; enabled by the formal structure of knowledge graphs.

Inter-rater Reliability

A statistical measure of agreement between different authors (human or computational) assessing the same content; commonly measured using Cohen's kappa or Fleiss' kappa.

Intra-rater Reliability

A measure of consistency for an individual contributor over time; used to detect drift or changing interpretation.

Language Coordinate

A coordinate type specifying which natural languages, dialects, and description types are included in a view.

Logic Coordinate

A coordinate type specifying description logic configuration, including reasoner selection and stated vs. inferred relationship handling.

Navigation Coordinate

A coordinate type specifying how the knowledge graph is traversed and presented, including hierarchy selection and root concepts.

Patterns

Component type that defines structure; templates for organizing information in the minimal component model.

Primordial Starter Set

The foundational set of concepts and patterns upon which all other Starter Sets build; includes identity, versioning infrastructure, and primitive types.

Reasoner

A description logic engine that computes logical consequences from stated axioms; used for consistency validation and inferring relationships in knowledge graphs.

Reification

The process of making an abstraction concrete by anchoring it to a Concept within the knowledge graph, giving it permanent identity and enabling semantic attachment.

Reified View Coordinate

A View Coordinate specification that has been anchored to a Concept, providing permanent identity, enabling descriptions and release lifecycle semantics to be attached, and making the configuration queryable and referenceable.

Semantic Merge

The process of combining content from two paths with validation based on logical consistency rather than textual comparison; enabled by explicit component identity.

Semantics

Component type that adds meaning by extending concepts; enables knowledge to emerge without inventing new basic types.

STAMP Coordinate

A coordinate type providing five dimensions (Status, Time, Author, Module, Path) for identifying component state.

Stamps

Component type representing version coordinates; embodies the five STAMP dimensions.

Starter Set

A coherent collection of concepts, patterns, and infrastructure that provides foundation or domain capabilities; Starter Sets compose hierarchically.

Statistical Process Control

The application of statistical methods to monitor and control a process; in IKE-Flow, used to detect quality drift and calibrate review intensity based on measured reliability.

View Coordinate

A complete configuration specification integrating STAMP, Language, Logic, Navigation, and potentially future coordinate types; can be used transiently or reified for persistence.

Withdrawal

The process of marking published content as erroneous such that it is treated as if it should never have existed, while preserving audit trails.

15. References

Bibliography

- [gitflow] Vincent Driessen. "A successful Git branching model." January 5, 2010. <https://nvie.com/posts/a-successful-git-branching-model/>
- [semver] Tom Preston-Werner. "Semantic Versioning 2.0.0." <https://semver.org/>
- [maven] Apache Software Foundation. "Apache Maven Project." <https://maven.apache.org/>
- [owl2-profiles] W3C. "OWL 2 Web Ontology Language Profiles (Second Edition)." December 11, 2012. <https://www.w3.org/TR/owl2-profiles/>
- [owl2-overview] W3C. "OWL 2 Web Ontology Language Document Overview (Second Edition)." December 11, 2012. <https://www.w3.org/TR/owl2-overview/>
- [snomed-tig] SNOMED International. "SNOMED CT Technical Implementation Guide." <https://confluence.ihtsdotools.org/display/DOCTIG>
- [snomed-rf2] SNOMED International. "RF2 Release Format." <https://confluence.ihtsdotools.org/display/DOCRELFMT>
- [loinc-guide] Regenstrief Institute. "LOINC User's Guide." <https://loinc.org/guides/>
- [fhir-lifecycle] HL7 International. "FHIR Resource Lifecycle." <https://www.hl7.org/fhir/lifecycle.html>
- [fhir-codesystem] HL7 International. "FHIR CodeSystem Resource." <https://www.hl7.org/fhir/codesystem.html>
- [bpmn] Object Management Group. "Business Process Model and Notation (BPMN) Version 2.0.2." December 2013. <https://www.omg.org/spec/BPMN/2.0.2/>
- [dublin-core] Dublin Core Metadata Initiative. "DCMI Metadata Terms." <https://www.dublincore.org/specifications/dublin-core/dcmi-terms/>
- [cohen-kappa] Cohen, Jacob. "A coefficient of agreement for nominal scales." *Educational and Psychological Measurement* 20, no. 1 (1960): 37-46. <https://doi.org/10.1177/001316446002000104>
- [montgomery-spc] Montgomery, Douglas C. *Introduction to Statistical Quality Control*. 7th ed. Hoboken, NJ: Wiley, 2012. ISBN: 978-1118146811.

16. Document History

Version	Date	Changes
0.1	2025-01	Initial draft
0.2	2025-01	Added GitFlow lessons learned; introduced Named Stamp Coordinates; comprehensive withdrawal alternatives; Starter Set architecture; Mermaid diagrams
0.3	2025-01	Added knowledge graph advantages section; scalable curation architecture with statistical process control; quality metrics; AI-assisted authoring considerations; PNG diagram format
0.4	2025-01	Reframed from "knowledge graphs require" to "knowledge graphs enable"; emphasized explicit structure as advantage enabling scalable curation; positioned formal semantics as foundation for automation
0.5	2025-01	Replaced "classifier" with "reasoner"; replaced "quality gates" with "consistency validation"; expanded from terminology to knowledge curation; added minimal component model section; added comprehensive references with hyperlinks; improved diagram readability; clarified IKE acronym
0.6	2025-01	Introduced View Coordinates integrating multiple coordinate types (STAMP, Language, Logic, Navigation); replaced Named Stamp Coordinate with Reified View Coordinate; added release lifecycle through semantic attachment; single-repository release promotion model

[1] IKE stands for **Integrated Knowledge Exchange**, a project focused on enabling seamless integration and exchange of structured knowledge across systems and organizations. The "-Flow" suffix follows the naming convention established by GitFlow, indicating a workflow model built on these foundations.

[2] Vincent Driessen, "A successful Git branching model," January 5, 2010. <https://nvie.com/posts/a-successful-git-branching-model/>

[3] The STAMP-versioned knowledge graph model uses a deliberately minimal set of component types—Concepts, Patterns, Semantics, and Stamps—enabling flexibility and consistency. Semantics add meaning to the items they extend, allowing knowledge to emerge without inventing new basic types. Fields within Semantics are dynamically defined through concepts representing data type, meaning, and purpose, enabling semantic integration and reasoning over new content without compile cycles.

[4] Apache Software Foundation, "Apache Maven Project," <https://maven.apache.org/>

[5] Jacob Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement* 20, no. 1 (1960): 37-46. <https://doi.org/10.1177/001316446002000104>

[6] Douglas C. Montgomery, *Introduction to Statistical Quality Control*, 7th ed. (Hoboken, NJ: Wiley, 2012). ISBN: 978-1118146811

[7] W3C, "OWL 2 Web Ontology Language Profiles (Second Edition)," December 11, 2012. https://www.w3.org/TR/owl2-profiles/#OWL_2_EL

[8] SNOMED International, "SNOMED CT Technical Implementation Guide," <https://confluence.ihtsdotools.org/display/DOCTIG>

[9] Regenstrief Institute, "LOINC User's Guide," <https://loinc.org/guides/>

- [10] Object Management Group, "Business Process Model and Notation (BPMN) Version 2.0.2," December 2013. <https://www.omg.org/spec/BPMN/2.0.2/>
- [11] HL7 International, "FHIR Resource Lifecycle," <https://www.hl7.org/fhir/lifecycle.html>
- [12] W3C, "OWL 2 Web Ontology Language Document Overview (Second Edition)," December 11, 2012. <https://www.w3.org/TR/owl2-overview/>
- [13] Dublin Core Metadata Initiative, "DCMI Metadata Terms," <https://www.dublincore.org/specifications/dublin-core/dcmi-terms/>
- [14] SNOMED International, "RF2 Release Format," <https://confluence.ihtsdotools.org/display/DOCRELFMT>
- [15] HL7 International, "FHIR CodeSystem Resource," <https://www.hl7.org/fhir/codesystem.html>